

Plan de Trabajo Kanban y Arquitectura de Software

Ecosistema "Technical Bridge" & "DataBridge"

PROYECTO CAPSTONE

Documento Técnico Definitorio de Alcance, Contexto y Flujo Continuo

1. Contexto del Proyecto (Capstone)

Este proyecto se desarrolla en el marco de un **Proyecto Capstone** (proyecto de título/integrador), cuyo propósito es demostrar el dominio avanzado de ingeniería de software, arquitectura de sistemas y gestión de flujos de trabajo profesionales. El problema central a resolver es la ineficiencia, falta de transparencia y fricción en los procesos de cobranza y pago de deudas entre instituciones (acreedores) y usuarios finales (deudores).

El ecosistema tecnológico propuesto actúa como un intermediario digital que automatiza la conciliación, flexibiliza las opciones de pago mediante repactación dinámica y elimina barreras de entrada tecnológicas (como el uso de contraseñas tradicionales), todo ello soportado por una arquitectura escalable de nivel empresarial.

2. Resumen Ejecutivo del Ecosistema

La solución se estructura en dos plataformas independientes pero interconectadas (B2B2C):

- **Technical Bridge (Core B2C):** Plataforma orientada al deudor final. Implementa autenticación *passwordless* (Magic Links vía token de email), un motor de repactación interactivo, integración con pasarelas de pago (Mercado Pago, Khipu, Webpay) y asistencia automatizada mediante IA (chatbot NLP).
- **DataBridge (Core B2B):** Plataforma que simula al cliente corporativo (acreedor). Su función es inyectar cargas masivas de datos (CSV) hacia Technical Bridge y proveer un dashboard analítico para monitorear el estado de la cartera, métricas de recuperación y conciliaciones.

3. Arquitectura del Sistema

Se adopta un patrón de Microservicios para el backend (desarrollado en Java Spring Boot y Python) y una Single Page Application (SPA) para el frontend (desarrollado en React). La comunicación entre módulos críticos se maneja de forma asíncrona para garantizar alta disponibilidad.

Desglose de Componentes:

- **API Gateway (Spring Cloud Gateway):** Punto único de entrada. Enruta el tráfico, maneja CORS y aplica políticas de Rate Limiting.
- **MS-Auth:** Microservicio de seguridad. Genera tokens UUID de un solo uso, despacha correos (SMTP) y emite/valida JSON Web Tokens (JWT).
- **MS-Debt:** Motor transaccional relacional (SQL Server/Oracle). Administra el saldo, ejecuta el algoritmo de repactación de cuotas y mantiene el registro inmutable de auditoría.
- **MS-Payments:** Interfaz con pasarelas externas. Expone webhooks seguros, verifica firmas criptográficas de transacciones y publica eventos de éxito.
- **MS-AI:** Microservicio en Python (FastAPI/Flask) especializado en Procesamiento de Lenguaje Natural para el chatbot y análisis de sentimiento.
- **Message Broker (RabbitMQ):** Bus de eventos. Permite que *MS-Payments* notifique a *MS-Debt* sobre un pago completado sin generar acoplamiento síncrono, protegiendo la consistencia de los datos.

4. Metodología de Trabajo: Kanban

Para la ejecución del proyecto Capstone, se abandona el enfoque de Sprints cerrados en favor de la metodología **Kanban**, orientada al flujo continuo de valor y a la minimización de cuellos de botella (bottlenecks). El desarrollo se regirá por los siguientes principios:

- **Flujo Visual:** Las tareas transitarán por un tablero con los estados: *Backlog*, *To Do (Ready)*, *In Progress*, *Code Review / Testing*, *Done*.
- **Límites WIP (Work In Progress):** Se establecerá un límite estricto de tareas simultáneas en la columna "In Progress" para asegurar que los desarrollos se terminen (Done) antes de iniciar nuevos requerimientos.
- **Despliegue Continuo:** Las funcionalidades pasarán a integración una vez superen la revisión de código, sin esperar al final de un ciclo temporal fijo.

5. Definición del Backlog (Épicas y Tareas)

A continuación se detallan las unidades de trabajo técnico que alimentarán la columna "To Do" del tablero Kanban, agrupadas por su dominio funcional (Épicas).

Épica (Dominio)	Backend & Infraestructura (Work Items)	Frontend React & DataBridge (Work Items)
1. Core y Autenticación (MS-Auth & Gateway)	- Configurar API Gateway con enrutamiento. - Crear MS-Auth: Endpoint de generación UUID y envío SMTP. - Implementar Rate Limiting (Bucket4j). - Validar token y emitir JWT.	- Inicializar repositorio React (Zustand, Axios, Tailwind). - Maquetar interfaz de Login "Passwordless". - Implementar captura de UUID desde URL y guardado de sesión JWT.
2. Dominio de Deuda (MS-Debt & DB)	- Modelar esquema relacional (SQL). - Desarrollar MS-Debt: CRUD de deudas y tabla de cuotas. - Lógica matemática de repactación (fechas y montos). - API de ingesta masiva CSV para DataBridge.	- Technical Bridge: Vista principal "Mis Deudas". - Crear componente Simulador de Repactación (Slider de meses). - DataBridge: UI para subida de archivo CSV (Drag & Drop).
3. Flujo de Pagos (MS-Payments)	- Desarrollar MS-Payments: Conexión API Mercado Pago/Khipu. - Crear Endpoints para recibir Webhooks. - Levantar RabbitMQ y configurar publicación de pago_exitoso. - MS-Debt: Listener de RabbitMQ para actualizar estado a PAGADA.	- Integrar botón "Pagar" que abra la pasarela (Checkout Pro o Link). - Manejar UI de estado pendiente y *polling* o web sockets para reflejar el pago exitoso sin recargar.
4. Inteligencia Artificial (MS-AI & NLP)	- Levantar entorno Python/FastAPI. - Crear endpoint de consulta para NLP (Chatbot). - Lógica de lectura segura (Read-Only) a la DB para informar saldo en el chat.	- Construir widget de Chatbot flotante en la UI del deudor. - Manejar historial de mensajes en el estado local de React.

Épica (Dominio)	Backend & Infraestructura (Work Items)	Frontend React & DataBridge (Work Items)
5. Analítica B2B (DataBridge Dashboard)	- Crear endpoints de agregación en MS-Debt (total recaudado, deudas activas). - MS-Debt: Librería para autogenerar PDF "Certificado de Deuda Cero".	- DataBridge: Maquetar Dashboard Analítico usando Recharts/Chart.js. - Technical Bridge: Interfaz para descargar el Certificado PDF.